



8. Cross-Validation in Neural Networks

"Don't Just Fit, Generalize."

Previous Section:

 [7. Neural Networks: Regression](#)

Contents:

[1. Overview of Cross-Validation](#)

[2. Cross-Validation for Classification Problems](#)

[3. Cross-Validation for Regression](#)

[Summary](#)

[References](#)

Training a Neural Network using epochs sounds reasonable at first. You feed the model data, let it learn repeatedly over multiple epochs, evaluate the loss, and eventually hope the model generalizes well. But here is the uncomfortable reality:

Sometimes, a single train-validation split is simply not enough. Because what if the validation set just happens to be “easy”? Or worse, what if the model performs well only because of a fortunate split in the data?

In Deep Learning, this becomes especially dangerous when datasets are limited. A model may appear highly accurate during training, yet completely fail when exposed to new real-world data.

So instead of trusting one split blindly, we introduce a more reliable strategy: Cross-Validation.

The core idea is simple: Rather than evaluating the model once, we evaluate it multiple times across different portions of the dataset. This allows us to test whether the model truly understands the underlying patterns... or whether it merely became lucky with one particular validation set.

You can think of Cross-Validation as repeatedly asking the Neural Network the same question from different perspectives. If the model performs consistently well across all splits, we gain confidence that it has genuinely learned the data instead of memorizing specific examples.

And honestly, this is one of the most important mindset shifts in Machine Learning and Deep Learning: A good model is not one that performs well once. A good model is one that performs well consistently.

1. Overview of Cross-Validation

We actually encountered this concept before in Machine Learning. So here, I'll briefly revisit the idea and shift it toward the perspective of Deep Learning.

At its core, Cross-Validation is simply a technique used to divide the dataset into multiple validation sets in order to evaluate the model more reliably. The most common approach is: K-Fold Cross-Validation

The idea is straightforward. Suppose we have 1000 training samples and 200 testing samples. The testing set is usually left untouched until the very end because it represents completely unseen data. So during training, our main focus becomes the training set itself.

But here is the issue: If we only split the training data once into: training subset and validation subset, then the evaluation may become biased. Maybe the validation set is unusually easy. Maybe the split accidentally favors the model. Or maybe the validation data does not represent the overall dataset very well.

So instead of relying on one validation split, Cross-Validation repeatedly creates multiple validation sets from the training data. You can think of the validation set as a "mock exam" during training. The model trains on one portion of the data, then gets evaluated on another unseen portion. After that, the roles rotate. This process continues multiple times until every portion of the dataset has been used as validation at least once.

For example, with $K = 5$. In Cross-Validation, the training data is divided into 5 folds. 4 folds are used for training. 1 fold is used for validation. Then the validation fold rotates. So the model is evaluated 5 different times.

This gives us a much better understanding of whether the model truly generalizes well or merely performs well on one fortunate split.

In Machine Learning, Cross-Validation is extremely common because datasets are often smaller and models train relatively quickly. In Deep Learning, however, things become more complicated. Neural Networks are computationally expensive. Training one model already takes time. Training the same model repeatedly across multiple folds can become very costly.

That is why Deep Learning sometimes uses simplified validation approaches instead of full K-Fold Cross-Validation, especially for massive datasets. Still, the philosophy remains the same: We do not trust a model simply because it performs well once. We trust it when it performs consistently across different subsets of data.

2. Cross-Validation for Classification Problems

Now let's finally apply Cross-Validation to a real Neural Network classification problem. We will use one of the most famous beginner datasets in Machine Learning: The Iris Dataset

The task is simple: Given flower measurements such as sepal length, sepal width, petal length, and petal width. The Neural Network must classify the flower into one of three species. And instead of using a single validation split, we will manually implement: $K = 5$

This is K-Fold Cross-Validation, which means the model will train 5 separate times, each time using a different portion of the dataset as validation data.

Step 1: Build the Neural Network - First, we define the architecture.

```
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential

def build_classification_model():
```

```

hidden_layer1 = layers.Dense(64, activation='relu')
hidden_layer2 = layers.Dense(32, activation='relu')
output_layer = layers.Dense(3, activation='softmax')
model = Sequential([hidden_layer1, hidden_layer2, output_layer])
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
return model

```

- **Hidden Layer 1** has 64 neurons, uses ReLU activation, learns nonlinear relationships
- **Hidden Layer 2** has 32 neurons, acts as a deeper feature extractor
- **Output Layer** has 3 neurons. Because the Iris dataset contains three labels (Iris Setosa, Iris Versicolor, Iris Virginica)
- The Softmax activation converts outputs into probabilities. Example: `[0.1, 0.7, 0.2]` . Meaning 10% Setosa, 70% Versicolor, 20% Virginica

Step 2 — Load and Prepare the Dataset

- **Load the Dataset**

```

from sklearn.datasets import load_iris
iris = load_iris()

X = iris.data
y = iris.target

```

Here: `X` contains the flower features and `y` contains the class labels

- **One-Hot Encoding**

```

from tensorflow.keras.utils import to_categorical

y = to_categorical(y, num_classes=3)

```

The labels originally look like this: 0, 1, 2. But categorical crossentropy expects: [1,0,0], [0,1,0], [0,0,1] → This process is called: One-Hot Encoding

- **Standardize the Features**

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

This rescales the features so they have: mean = 0 and standard deviation = 1. Very important for Neural Networks because large feature differences can destabilize training.

- **Train-Test Split**

```
train_data, test_data, train_targets, test_targets = train_
test_split(X_scaled, y,

test_size=0.2, random_state=42)
```

This keeps: 80% for training and 20% for final testing. The testing set remains untouched during Cross-Validation.

Step 3 — Apply K-Fold Cross-Validation Manually

Now comes the important part.

```
K = 5
num_val_samples = len(train_data) // K
num_epochs = 80
classification_histories = []
```

We split the training data into: 5 folds. Then we compute how many samples each fold should contain. The model trains for 80 epochs during each fold.

Step 4 — The Cross-Validation Loop

```

for i in range(K):
    print('processing fold #', i + 1)
    val_data = X_scaled[i * num_val_samples: (i + 1) * num_val_samples]
    val_targets = y[i * num_val_samples: (i + 1) * num_val_samples]

    partial_train_data = np.concatenate(
        [train_data[:i * num_val_samples],
         train_data[(i + 1) * num_val_samples:]],
        axis=0)

    partial_train_targets = np.concatenate(
        [train_targets[:i * num_val_samples],
         train_targets[(i + 1) * num_val_samples:]],
        axis=0)

    model = build_classification_model()
    history = model.fit(
        partial_train_data,
        partial_train_targets,
        epochs=num_epochs,
        batch_size=16,
        verbose=0)

    classification_histories.append(history.history['accuracy'])

average_accuracy = np.mean(classification_histories, axis=0)
print('Average accuracy:', average_accuracy)

```

```

processing fold # 1
processing fold # 2
processing fold # 3
processing fold # 4
processing fold # 5

```

```
Average accuracy: 0.9230208367854357
```

This section selects the current validation fold.

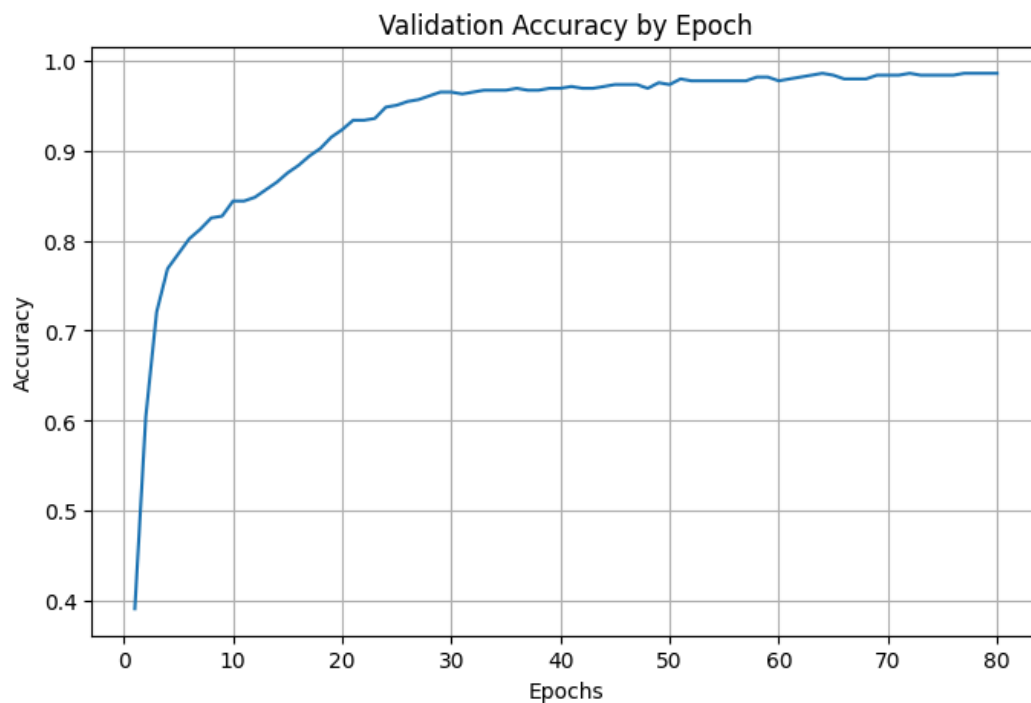
- For each iteration: one fold becomes validation data and the remaining folds become training data.
- The **Partial Training Data** is used to remove the validation fold from the training data. So the model never trains on the current validation fold.
- The **Partial Training Targets** is used for labels
- We train the model with the defined epoch and `batch_size=16`
- Each fold then stores its training accuracy history. After all folds finish, we average them together.

So after training across: $80 \text{ epochs} \times 5 \text{ folds}$, the Neural Network achieves roughly: 92.3% accuracy

Step 5 — Visualize Accuracy Across Epochs

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.plot(range(1, num_epochs + 1), average_accuracy)
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Validation Accuracy by Epoch')
plt.grid(True)
plt.show()
```

When observing the graph, we usually notice something interesting. The validation accuracy improves rapidly during the earlier epochs. Then eventually, the improvement slows down, the curve begins to flatten, performance stagnates

In this example, the accuracy improves significantly before around *epoch* ≈ 30 . After that, the changes become very small. This tells us something important: The Neural Network already learned most meaningful patterns relatively early. Beyond that point, additional epochs provide diminishing returns, training longer may not significantly improve performance, excessive training could even risk overfitting.

And honestly, this is one of the reasons Cross-Validation is so useful in Deep Learning. It helps us observe whether the learning behavior remains consistent across multiple subsets of data instead of trusting one fortunate validation split.

3. Cross-Validation for Regression

Earlier, we used Cross-Validation for a classification problem. Now let's move toward something slightly different: Regression. Instead of predicting categories or labels, regression predicts continuous numerical values.

In this example, we will use the famous Boston Housing Dataset. The goal is simple: Given information about houses and neighborhoods, the Neural

Network must predict housing prices.

And once again, instead of trusting one validation split, we will apply:K-Fold Cross-Validation with $K = 5$.

Step 1 — Load the Dataset

```
from tensorflow.keras.datasets import boston_housing
from tensorflow import keras

(train_data, train_targets), (test_data, test_targets) = boston_housing.load_data()
```

Here: `train_data` contains house features and `train_targets` contains house prices

Unlike classification, the target values are continuous numbers. Example: `21.5`, `34.2`, `15.8`. So this becomes a regression problem.

Step 2 — Normalize the Data

Neural Networks are highly sensitive to feature scales. Some features may contain small values: `0.02`, while others may contain much larger ones: `400`

This can destabilize training. So we normalize the dataset.

```
mean = train_data.mean(axis=0)
std = train_data.std(axis=0)

train_data -= mean
test_data -= mean

train_data /= std
test_data /= std
```

This transformation gives the features: `mean=0` and `standard\ deviation=1`, which helps the Neural Network train more efficiently.

Step 3 — Build the Regression Model

```

from tensorflow.keras import layers
from tensorflow.keras.models import Sequential

def build_regression_model():
    hidden_layer1 = layers.Dense(64, activation='relu')
    hidden_layer2 = layers.Dense(64, activation='relu')
    output_layer = layers.Dense(1)
    model = keras.Sequential([hidden_layer1, hidden_layer2,
                               output_layer])

    model.compile(optimizer='rmsprop', loss='mse', metrics=
['mae'])
    return model

```

- **Hidden Layers:** These layers learn nonlinear relationships between housing features and prices.
- **Output Layer:** There is only 1 output neuron. Because regression predicts a single continuous numerical value. Unlike classification, there is no activation function Softmax, and there are no class probabilities. The model directly outputs a number (like 24.7), meaning predicted house price
- The model uses **Mean Squared Error (MSE), which** heavily penalizes large prediction errors. Used as the training loss.
- The model uses **Mean Absolute Error (MAE), which** measures the average prediction error magnitude. Much easier to interpret. Example: MAE = 2.3 means on average, the predictions are off by about 2.3 units.

Step 4 — Apply K-Fold Cross-Validation + Loops

```

import numpy as np

k = 5
num_val_samples = len(train_data) // k
num_epochs = 100
all_mae_histories = []

for i in range(k):

```

```

print('processing fold #', i + 1)

# Set validation fold
val_data = train_data[i * num_val_samples: (i + 1) * num_val_samples]
val_targets = train_targets[i * num_val_samples: (i + 1) * num_val_samples]

# Built partial train
partial_train_data = np.concatenate(
    [train_data[:i * num_val_samples],
     train_data[(i + 1) * num_val_samples:]],
    axis=0)

partial_train_targets = np.concatenate(
    [train_targets[:i * num_val_samples],
     train_targets[(i + 1) * num_val_samples:]],
    axis=0)

# Build regression model
model = build_regression_model()

# Train and save history
history = model.fit(
    partial_train_data,
    partial_train_targets,
    validation_data=(val_data, val_targets),
    epochs=num_epochs,
    batch_size=16,
    verbose=0)

# Take mae history of current fold and append to all_mae_histories
mae_history = history.history['val_mae']
all_mae_histories.append(mae_history)

# Compute average_mae_history

```

```
average_mae_history = [np.mean([x[i] for x in all_mae_histories]) for i in range(num_epochs)]
```

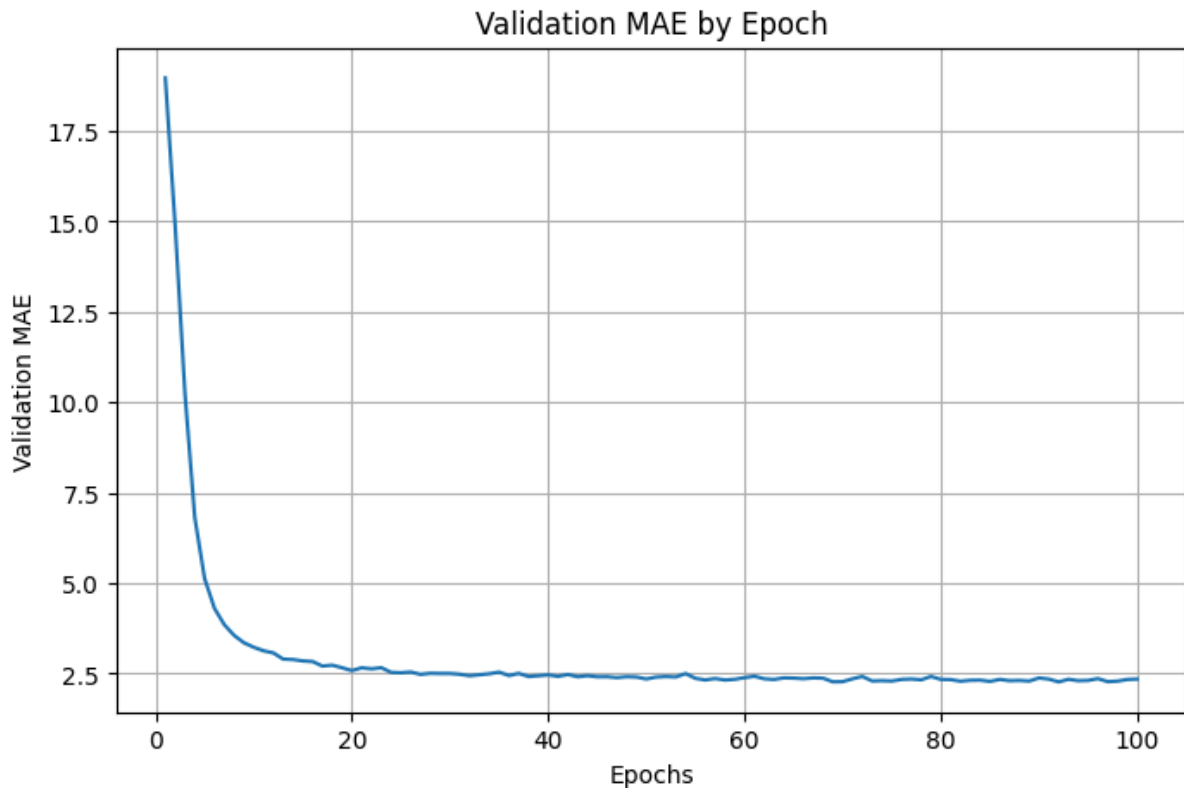
We divide the training data into 5 folds and train the model repeatedly across different validation splits.

- **Build Partial Training Data:** This removes the validation fold from training.
- **Build Partial Training Targets:** Same process for the targets.
- The important part here is: `validation_data=(val_data, val_targets)` . Now the model evaluates validation performance during every epoch.
- Each fold then stores its validation MAE history.
- Finally, computes the average validation MAE across all folds. Instead of trusting one fold, we average everything together.

Step 5 — Visualize Validation MAE

```
import matplotlib.pyplot as plt

# Plot average_mae_history
plt.figure(figsize=(8, 5))
plt.plot(range(1, len(average_mae_history) + 1), average_mae_history)
plt.xlabel('Epochs')
plt.ylabel('Validation MAE')
plt.title('Validation MAE by Epoch')
plt.grid(True)
plt.show()
```



At the beginning, the validation MAE decreases rapidly. This means the model is learning meaningful patterns. But eventually, something interesting happens. Starting around *epoch* ≈ 20 , the MAE begins to stagnate. The curve stops improving significantly.

This tells us that the Neural Network already learned most useful relationships, additional epochs provide smaller improvements, continuing too long may risk overfitting

And honestly, this is one of the hidden purposes of Cross-Validation. Not just measuring accuracy. But understanding: ***How the model learns over time.***

Summary

This entire section is actually about something more subtle. Something that is not really “Deep Learning” itself. Cross-Validation does not make Neural Networks deeper. It does not invent new architectures. It does not magically improve intelligence.

Instead, it teaches something more important: **How to trust your model.**

Because one successful training run means very little. A Neural Network may perform well because the split was fortunate, the validation set was easy,

randomness favored the model

Cross-Validation exists to reduce that illusion. It repeatedly tests the model across different subsets of data to see whether the learning behavior remains consistent. And honestly, this is one of the biggest transitions from beginner Machine Learning toward serious model evaluation: We stop asking ***"Did the model work once?"***. And start asking: ***"Does the model work reliably?"***

References

<https://www.geeksforgeeks.org/machine-learning/cross-validation-machine-learning/>

<https://www.kaggle.com/code/alexisbcook/cross-validation>

Next Section:

 [9. Evaluating & Improving Neural Network Models](#)